

Track 1 Practice Set

Tier 1 — Python Fundamentals

PY1 — Prescription label formatter

Given drug = "Amoxicillin", dose = 500, unit = "mg", freq = 3, print a clean label: Amoxicillin 500 mg — take 3 times daily. Use an f-string.

PY2 — Weight list, manual stats

weights = [58, 72, 65, 90, 47, 83] (kg). Without NumPy, compute and print mean, max, min, and the range (max-min). Use sum(), len(), max(), min().

PY3 — Drug → standard dose dictionary

Build doses = {"amoxicillin": 500, "paracetamol": 650, "ibuprofen": 400}. Ask the user for a drug name, .lower() it, and print its dose. Handle "not found" with .get().

PY4 — Flag over-limit doses (loop + conditional)

daily_doses = [400, 900, 650, 1200, 300] mg, max_safe = 800. Loop through, print each dose and whether it's SAFE or OVER LIMIT. Count how many are over.

PY5 — BMI function with validation

Write def bmi(weight_kg, height_m): that returns BMI rounded to 1 decimal, and handles the case where height is 0 (avoid divide-by-zero). Test with 3 patients.

PY6 — Cohort filter (functions + list)

Given a list of patient dicts [{"id":1,"age":45,"bmi":31}, ...] (make 6), write a function that returns only patients with bmi >= 30.

Tier 2 — NumPy (arrays + vectorized thinking)

NP1 — Concentration array stats

conc = np.array([1.2, 3.5, 2.8, 4.1, 0.9, 3.3]) (mg/L). Print mean, std, median, and the index of the max value (np.argmax).

NP2 — Unit conversion, vectorized

Same array in mg/L. Convert the whole array to mcg/L (×1000) in one line, no loop. Then round to 0 decimals with np.round.

NP3 — Boolean masking (threshold cohort)

From conc, use a boolean mask to pull only values above 3.0. Then count how many with .sum() on the mask.

NP4 — 2D concentration-time matrix

Make a 4×3 array: 4 patients (rows) × 3 timepoints (columns). Compute the mean per patient (axis=1) and mean per timepoint (axis=0).

NP5 — np.where categorization

Given an age array, use `np.where` to label each as "adult" (≥ 18) or "minor". Then a nested `np.where` for three bins (child/adult/senior).

NP6 — Bridge to Pandas

Load ONE numeric column from the diabetes CSV using `np.genfromtxt(..., delimiter=',', skip_header=1, usecols=...)`. Compute its mean.

Tier 3 — Pandas (real datasets — the RWE core)

Dataset A: Pima Diabetes (P1–P6)

P1 — Load & inspect

`df = pd.read_csv("diabetes.csv")`. Run `.head()`, `.tail()`, `.shape`, `.info()`, `.describe()`. Write one sentence per method on what it told you.

P2 — Select & filter

Select just Glucose and Age. Then filter to patients with Glucose > 140. How many are there?

P3 (starred) — The “zero is really missing” problem

In this dataset, Glucose, BloodPressure, BMI, Insulin, SkinThickness have literal 0s that are impossible (a living person's BMI isn't 0). Count zeros per column, then replace them with NaN (`df[col].replace(0, np.nan)`), then count missing with `.isna().sum()`.

P4 — groupby

`df.groupby("Outcome").mean()`. Compare mean Glucose, BMI, Age between diabetic (1) and non-diabetic (0). Write the takeaway in one line.

P5 — Derived column (feature engineering)

Create `BMI_category` using `pd.cut` (bins: <18.5 underweight, 18.5–25 normal, 25–30 overweight, >30 obese). Then `value_counts()` on it.

P6 — Correlation

`df.corr()["Outcome"].sort_values()`. Which features correlate most with diabetes outcome? Name the top 2.

Dataset B: Drug Review (P7–P11) — messier, more realistic

P7 — Load & assess

It's tab-separated: `pd.read_csv("drugsComTrain_raw.tsv", sep="\t")`. Check `.shape`, `.info()`, `.isna().sum()`. This has ~160k rows — notice how `.head()` matters more when you can't eyeball everything.

P8 — value_counts (top drugs / conditions)

Top 10 most-reviewed `drugName`. Top 10 condition. Notice any junk values in condition (there are some malformed ones — flag them).

P9 (starred) — groupby + filter (the real analyst move)

Mean rating per `drugName`, BUT only for drugs with at least 50 reviews (otherwise averages are noise). Chain: `groupby` → `agg` (count & mean) → `filter` `count` ≥ 50 → `sort`.

P10 — Dates & trend

Parse the date column (pd.to_datetime), extract year (.dt.year), and count reviews per year. Is the data skewed to certain years?

P11 (starred) — Aggregate & export (portfolio-worthy)

Build a summary table: for the top 20 conditions, show number of drugs reviewed, total reviews, and mean rating. Export to condition_summary.csv.

Capstone

diabetes_eda.ipynb

One notebook that: loads the diabetes data → cleans the zero-missingness → shows group differences by outcome → engineers BMI category → states 3 findings in plain English.